PAC-MAN

Basics of Programming 3.

Project Specification

Balázs Tari

E3K6W5

October 16, 2026

Contents

1 Task Description **2**

2 Use-Cases **2**

3 Solution Overview **2**

4 Detailed Architecture and Class Structure **3**

5 Design Patterns **5**

6 Unit Testing **7**

7 I/O and Persistence Contract **8**

1 Task Description

Pac-Man is a two-dimensional desktop arcade game. The player controls the yellow figure in a labyrinth, whose task is to collect all dots while avoiding the ghosts moving around the map. If Pac-Man collects one of the 4 'Power Pellets' located in the labyrinth, the player gains the ability to eat ghosts for 6 seconds, earning additional points.

The software's graphical user interface is built using the Java Swing framework, the persistent storage of the game state is managed via file handling, and the code quality is ensured by automated JUnit unit tests.

2 Use-Cases

1. **Starting and controlling the game:** The user can start a new game from the menu and control Pac-Man within the labyrinth using the WASD keyboard keys.
2. **Collecting dots and victory:** As Pac-Man passes through the dots, they disappear, and the score increases. If all dots are consumed, the player wins and proceeds to the next level. At higher levels, the difficulty of the game increases.
3. **Collision detection with ghosts:** If the character collides with a ghost, it loses a life. If the number of lives reaches zero, the game ends (Game Over). Upon starting a new game, the player always begins at the first level.
4. **Saving high-scores:** At the end of a game, the system records and saves the achieved score to a file.
5. **Saving game state:** The game can be saved at any time and state, allowing the player to load and resume it later from a file.
6. **Level editor mode:** In addition to the base game, the user has the ability to design their own custom maps, save them, and load previously created maps to play on.

3 Solution Overview

1. **Applied technological solutions:** Java SE, Swing (JFrame, JPanel, Graphics for low-level drawing, Timer for the game loop), and JUnit 5 for automated testing.
2. **File formats:** A simple CSV file for storing the high-score table, and serialized objects or simple text matrices for storing map layouts and saved game states.

4 Detailed Architecture and Class Structure

Core Game Model

- **GameEngine:** The central controller of the game. It contains the game loop, updates the game state (positions, collisions), and manages the score, remaining lives, and the `currentLevel`. It is also responsible for saving and loading the active game state (utilizing `java.io.Serializable`). As the level increases, the duration of the ghosts' Frightened state progressively decreases to increase difficulty, while movement speeds remain constant.
- **Labyrinth:** The class representing the map. It stores the coordinates and states of walls, empty paths, and dots using a two-dimensional array.
- **GameObject** (Abstract Base Class): Encapsulates common properties for all moving entities (X and Y coordinates, speed, direction).
    - **PacMan** (Child Class): Represents the state of the player's character and handles its specific movement and input logic.
    - **Ghost** (Child Class): Represents the state of the ghosts and manages their pathfinding and movement strategies.

UI and Management Classes

- **MainFrame (JFrame):** The application's main window utilizing a CardLayout to switch between the menu, the active game, and the editor.
- **MainMenu (JPanel):** The start screen providing options to start a new game, load a saved game, view high scores, or open the map editor.
- **GamePanel (JPanel):** The active gameplay screen that bridges the Model and View. It receives the game state from the `GameEngine` and renders the maze and entities in its `paintComponent` method.
- **MapEditor (JPanel):** A graphical interface with a clickable grid allowing the user to draw custom mazes (placing walls, dots, and power pellets).
- **LevelManager (Factory):** A dedicated class responsible for serializing/saving the Labyrinth matrix to a file (CSV) and parsing it back into memory.

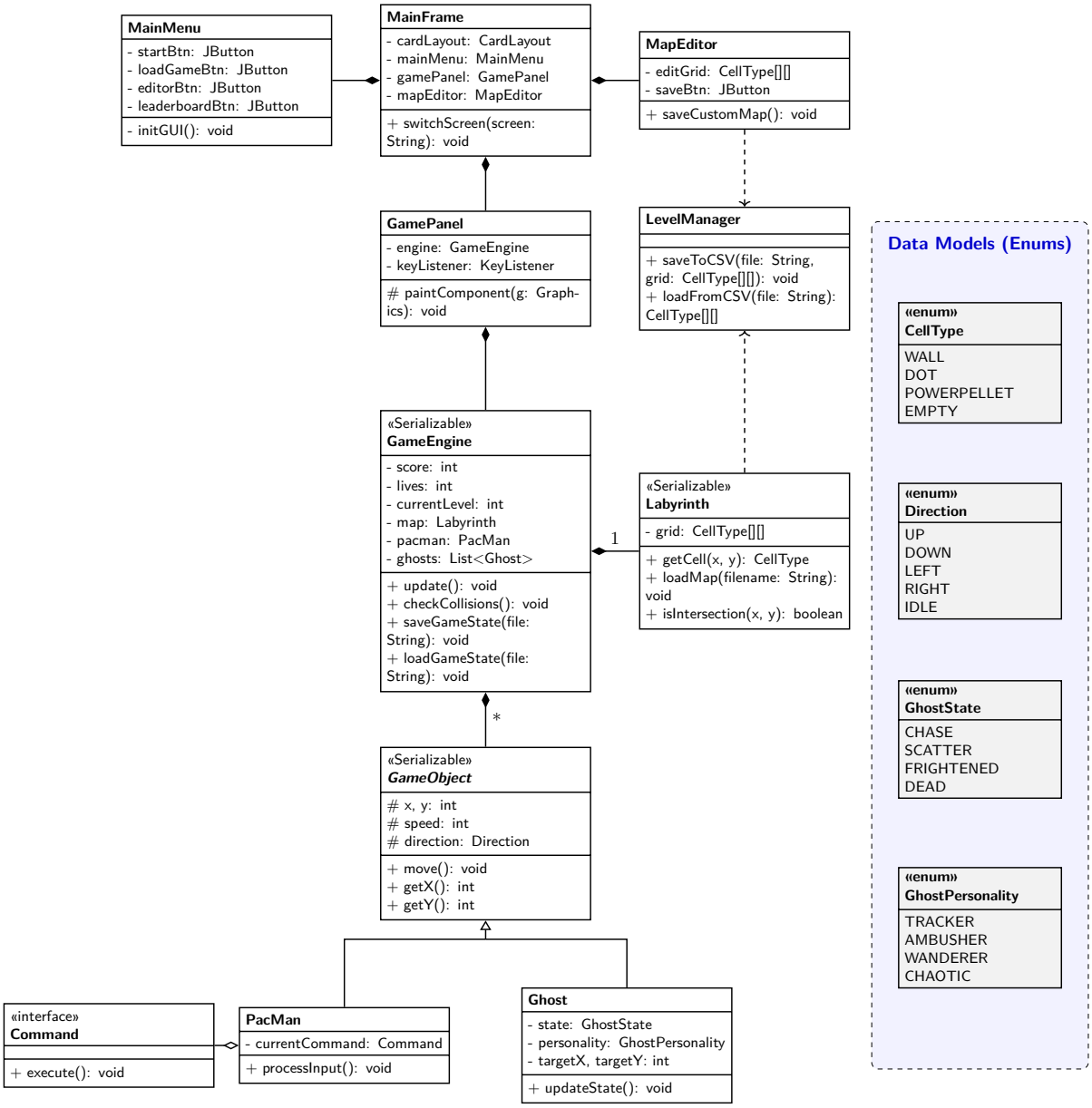


Figure 1: UML Class Diagram of the Complete Game Architecture

5 Design Patterns

To ensure the codebase meets university standards and demonstrates solid object-oriented principles, the implementation must explicitly utilize the following design patterns:

- **Ghost Movement:** The target-tile calculation and movement logic of the enemies is dynamically driven by the `GhostPersonality` enum. This serves as a lightweight Strategy Pattern: instead of creating numerous separate strategy classes that bloat the codebase, the ghost's behavior can be swapped at runtime simply by changing its active personality enum (as seen in the Chaotic ghost).

- **Ghost Behavior (Finite State Machine):** The ghosts' current mode is managed using a `GhostState` enum (`CHASE`, `SCATTER`, `FRIGHTENED`, `DEAD`). The transitions between these states are strictly governed by the game events shown in Figure 2.

  Ghost State Definitions:

  - **CHASE:** Each ghost uses its unique artificial intelligence to actively target and corner Pac-Man.
  - **SCATTER:** Ghosts temporarily stop pursuing Pac-Man and retreat to patrol their own designated corners of the maze.
  - **FRIGHTENED:** Ghosts turn blue, move slowly and randomly, and become vulnerable to being eaten after Pac-Man consumes a Power Pellet.
  - **DEAD (Eaten):** Reduced to just a pair of floating eyes, the ghost rushes back to the central Ghost House to respawn and regain its body.

- **Player Input (Command Pattern & Input Buffering):** Movement actions are encapsulated into concrete classes (e.g., `MoveUpCommand`) implementing a common `Command` interface with an `execute()` method. These objects are mapped to their corresponding WASD keys and buffered inside the `PacMan` instance (`currentCommand`). This explicitly solves the *corner pre-turning* (input buffering) mechanic, allowing for clean, dynamic execution at intersections without large switch-case statements.

- **Model-View-Controller Architecture:** The logical game state (Model) must be strictly separated from the Swing rendering (View). Logical entities like `PacMan` or `Ghost` must not contain any UI code, `javax.swing` imports, or `java.awt.Graphics` references.

- **Factory Pattern:** The creation of the labyrinth from a file (parsing a CSV/text file into the 2D array) must be handled by a dedicated `LevelFactory` or `MapLoader` class. This prevents the `GameEngine` from being bloated with initialization logic.

- **Observer Pattern:** The UI rendering component (`JPanel`) should observe the `GameEngine`. When the game state updates on a timer tick, the engine notifies the view to trigger a `repaint()`, decoupling the game loop from the graphics pipeline.

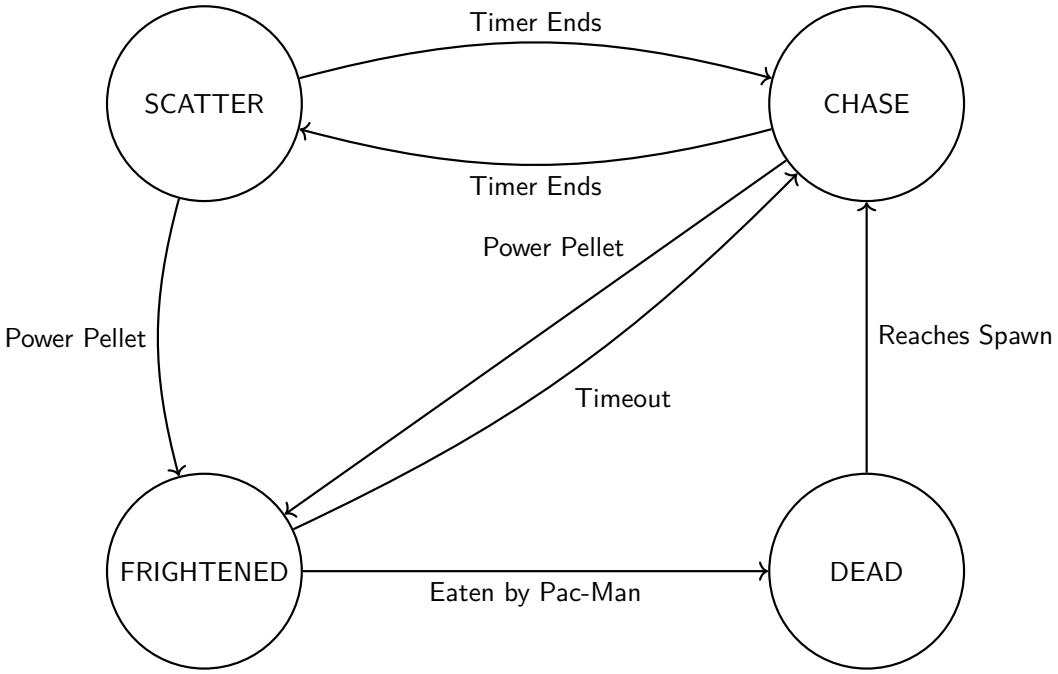


Figure 2: Finite State Machine (FSM) of Ghost Behaviors

6 Unit Testing

The project must include a minimum of **10 automated JUnit 5 tests**. These tests must strictly focus on the core logical components (the Model layer, such as `GameEngine` and `Labyrinth`) without instantiating the Swing graphical user interface.

Every test case must follow the industry-standard **Arrange-Act-Assert (Given-When-Then)** methodology:

- **Arrange (Given):** Initialize the `GameEngine`, load a controlled, minimalist test map matrix, and position the entities manually.
- **Act (When):** Trigger a specific game mechanic (e.g., execute a movement command, apply a timer tick, or force a collision).
- **Assert (Then):** Verify the expected changes in the game state using JUnit `Assertions` (e.g., checking if the score increased, coordinates changed, or ghost states updated).

Mandatory Test Scenarios

The test suite must cover critical gameplay mechanics, including but not limited to:

- **Wall Collision:** *Given* Pac-Man is adjacent to and facing a `WALL` tile, *When* a move command is processed, *Then* Pac-Man's (X, Y) coordinates must remain unchanged.
- **Dot Consumption:** *Given* Pac-Man moves onto a `DOT` tile, *When* the engine updates the state, *Then* the score must increase, and the tile must change to `EMPTY`.
- **Ghost Frightened Transition:** *Given* Pac-Man consumes a `POWERPELLET`, *When* the collision is resolved, *Then* all ghosts must transition from `CHASE/SCATTER` to the `FRIGHTENED` state.
- **Death Mechanic:** *Given* Pac-Man and a `CHASE` ghost occupy the same coordinate space, *When* collisions are evaluated, *Then* the player's life counter must decrease by 1, and the positions must reset to their respective spawn points.

7 I/O and Persistence Contract

The application is following these persistance protocols in order to ensure robust data handling:

1. High-Score Table

- **File:** `highscores.csv`
- **Responsibility:** Handled by the `LevelManager` class, which serves as the general I/O utility for the application.
- **Data Structure:** Each line must strictly follow the format: `PlayerName;Score;Date`.
- **Player Identity:** The system must not interrupt the player with a text input prompt to ask for their name. Instead, it must automatically fetch the identity of the user currently logged into the operating system using `System.getProperty("user.name")`.

2. Active Game State

Saving the complex state of an ongoing game (the exact position of Pac-Man, the current `GhostState` of every ghost, the remaining Frightened timer, the active level, and the state of every individual dot on the map) into a plain text file would require highly error-prone parsing.

Therefore, the active game state is saved using **Java Binary Serialization**:

- **File:** `savestate.dat`
- **Implementation:** All Core Model classes (`GameEngine`, `Labyrinth`, `GameObject`, `PacMan`, `Ghost`, and data Enums) must implement the `java.io.Serializable` interface.
- **Execution:** The `GameEngine.saveGameState()` method writes the entire engine instance to a binary file using an `ObjectOutputStream`. Loading a game simply reads the object back into memory using an `ObjectInputStream`, perfectly restoring the exact state without manual data mapping.

3. Custom Maps

- **Format:** Maps generated in the `MapEditor` are saved as raw `.csv` files.
- **Structure:** The file contains a comma-separated 2D matrix representing the `CellType` enum values (e.g., `WALL`, `EMPTY`, `DOT`), which is parsed line-by-line by the `LevelManager` when starting a new game.